

Data mining

Plan

- I- Introduction
- II- Méthodologie
- III- Outils et techniques de data mining
- IV- Méthodes de classification
 - ◆ Arbres de décision
 - ◆ k-plus-proches voisins
- V- Association
- VI- Segmentation
 - ◆ k-moyennes

I- Introduction

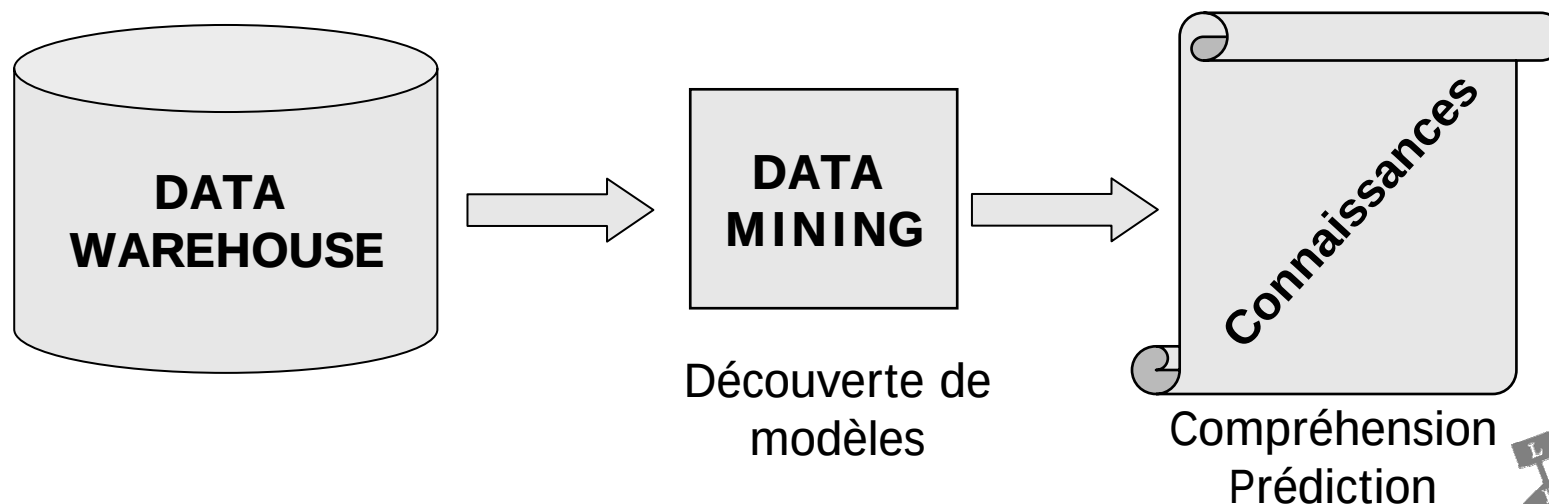
Exemple introductif

- Un client vient demander un crédit à une banque
- Le banquier cherche à savoir s'il va rembourser le crédit ou non
 - ◆ le client ne dira jamais qu'il ne va pas rembourser... mais le banquier peut inférer sur le comportement futur du client à partir de la base de données des anciens clients

Définition

- Data mining

- ◆ Ensemble de techniques d'exploration de données permettant d'extraire d'une base de données des connaissances sous la forme de modèles de description afin de décrire le comportement actuel et/ou de prédire le comportement futur des données



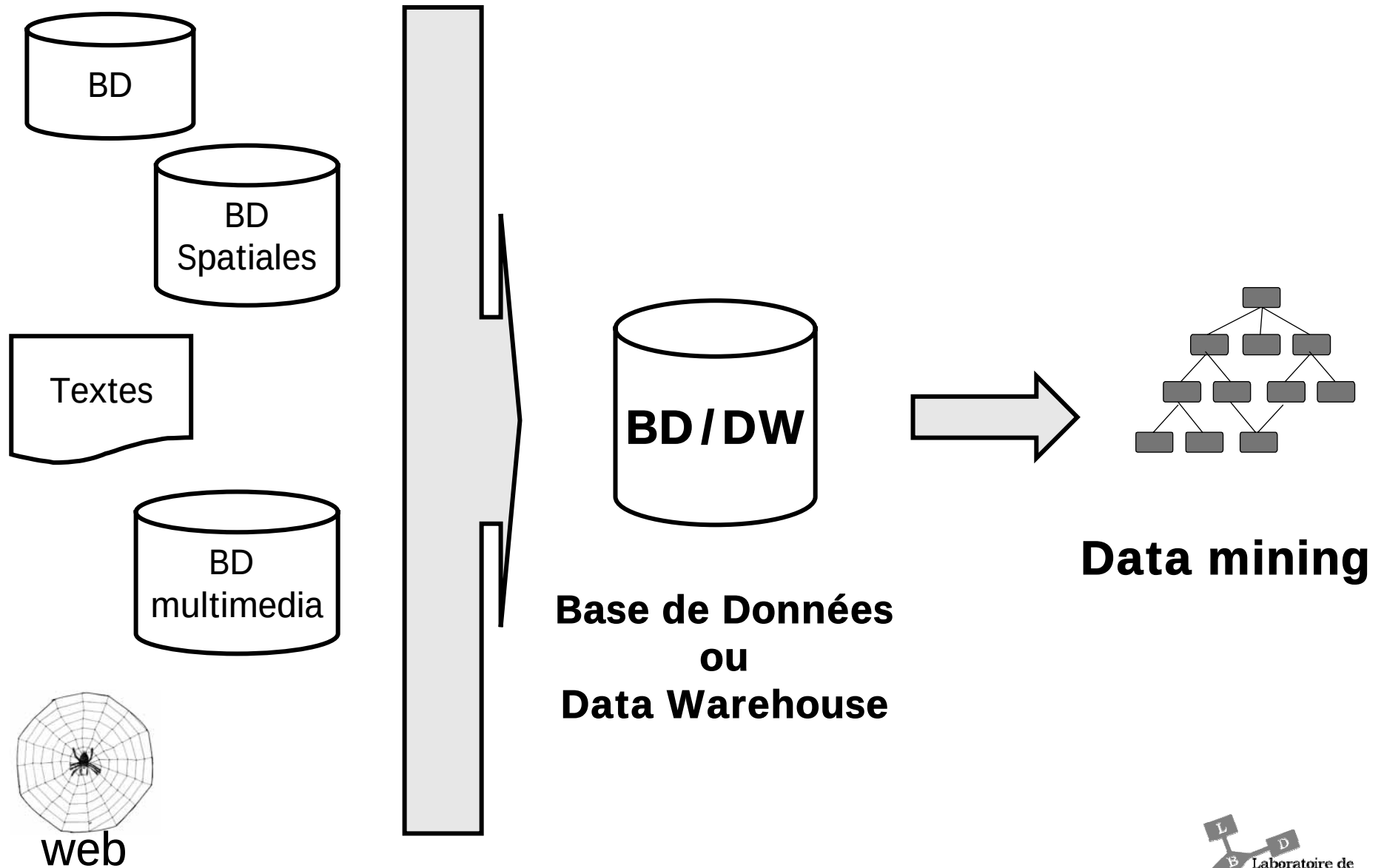
Connaissances

- Modèles de description peuvent être:
 - ◆ des analyses :
 - exemple: la distribution du trafic routier en fonction de l'heure
 - ◆ des règles:
 - par exemple: si un client n'a pas payé une facture > 100 000 Sfr alors il est dans 70% des cas en faillite
 - ◆ attribution des scores de qualité
 - par exemple, score de fidélité aux clients
 - ◆ classification des entités
 - par exemple, les mauvais payeurs

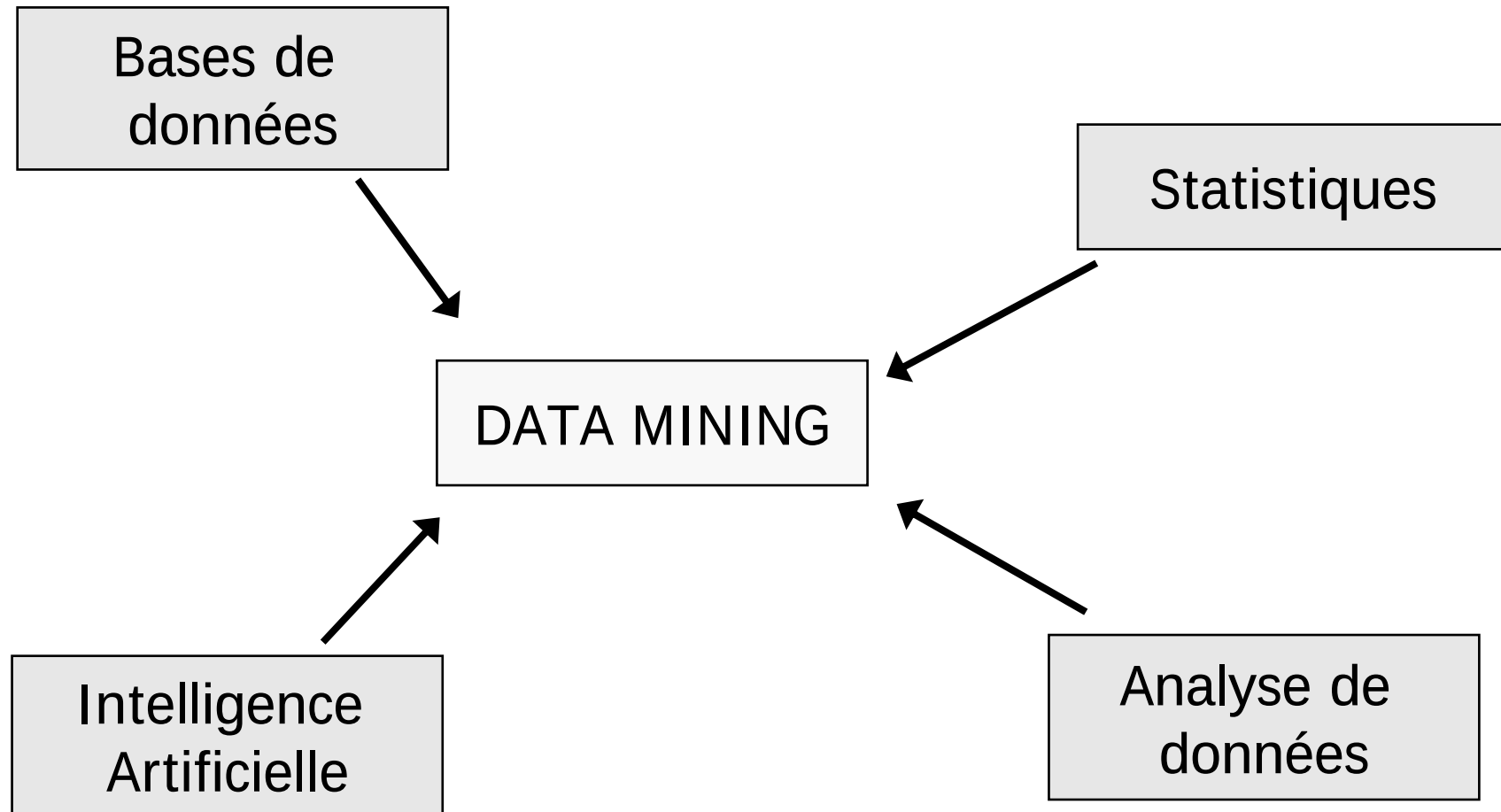
Domaines d'application

- Domaine des assurances
 - ◆ analyse des clients à "haut risque"
- Services financiers
 - ◆ Attribution d'un crédit bancaire
 - ◆ Détection de fraude
 - ◆ Marketing ciblé
- Grande distribution
 - ◆ Profil de consommateurs
 - ◆ Constitution des rayons
 - ◆ Marketing ciblé
- Médecine
 - ◆ Aide au diagnostic
- ...

Architecture



DM: Convergence de plusieurs disciplines



II - Méthodologie

Méthodologie

- 1. Identifier le problème
 - ◆ cerner les objectifs
- 2. Préparer les données
 - ◆ Collecter les données
 - ◆ Nettoyer les données
 - ◆ Enrichir les données
 - ◆ Codage, normalisation
- 3. Fouille des données
 - ◆ Choisir un type de modèle et une technique pour construire ce modèle
 - ◆ Validation
- 4. Utiliser le modèle

2. Préparer les données (i)

- Collecter les données
- Nettoyer les données
 - ◆ Points communs avec préparation des données pour la constitution d'un DW: si données provenant d'un DW, travail simplifié mais encore nécessaire pour
 - suppression des doublons, des erreurs de saisie
 - traitement des informations manquantes
- Enrichir les données
 - ◆ recours à d'autres données pour enrichir les données, ex: données médicales pour la banque
 - pb d'intégration de données

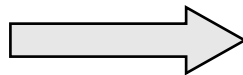
2. Préparer les données (ii)

- Codage, normalisation
 - ◆ Regroupements: attributs prenant un grand nombre de valeurs discrètes peuvent être regroupés
 - ◆ Exemple adresses en région

2. Préparer les données (iii)

- Codage, normalisation
 - ◆ Attributs discrets: codage des attributs discrets
 - par exemple magazine:

N°	Magazine	Adresse
1	BD	Lausanne
1	voiture	Lausanne
1	musique	Lausanne
2	BD	Morges
3	sport	Genève
4	sport	Ecublens



	Sport	BD	Voiture	Musique	Adresse
1	0	1	1	1	Vaud
2	0	1	0	0	Vaud
3	1	0	0	0	Genève
4	1	0	0	0	Vaud

2. Préparer les données (iv)

- Codage, normalisation (suite)
 - ◆ Changement de type: transformer les attributs dont le type n'est pas facilement utilisable pour des calculs (ex: date de naissance)

N°	Sport	BD	Voiture	Musique	Date naissance	Revenus	Adresse	Durée abonnement
1	0	1	1	1	50	70	Vaud	4
2	0	1	0	0	25	110	Valais	5
3	1	0	0	0	40	80	Genève	4
4	1	0	0	0	35	90	Vaud	null

2. Préparer les données (v)

- Codage, normalisation (suite)
 - ◆ Uniformisation d'échelle: même ordre de grandeur pour les attributs (ex: revenus /1000 pour être uniformisés avec date et durée)

N°	Sport	BD	Voiture	Musique	Date naissance	Revenus	Adresse	Durée abonnement
1	0	1	1	1	50	70	Vaud	4
2	0	1	0	0	25	110	Valais	5
3	1	0	0	0	40	80	Genève	4
4	1	0	0	0	35	90	Vaud	null

3. Fouille des données

- On a :
 - ◆ Identifié le problème
 - ◆ Préparé les données
- Maintenant: Fouille de données
 - ◆ Deux étapes:
 - Construction d'un modèle
 - Validation de ce modèle (erreur)

3.1 Construction du modèle

- Deux étapes:
 - ◆ Choix du type de modèle
 - Ex: Classification
 - ◆ Choix de la méthode pour construire ce modèle
 - Ex: arbres de décision
 - Choix d'un algorithme pour cette méthode : CART, C4.5, ...
- Processus itératif:
 - ◆ Nécessité d'explorer plusieurs modèles avant de trouver celui qui est adapté au besoin

3.2 Validation

- Deux types de validation:
 - ◆ Par un expert
 - ◆ Statistique
- Validation statistique
 - ◆ Décomposer les données en 2 ensembles:
 - un ensemble d'apprentissage: pour générer le modèle
 - un ensemble de test: pour tester le modèle sur un autre ensemble de données
 - ◆ "Cross-validation": ensemble divisé en n groupes:
 - $(n-1)$ ensembles d'apprentissage
 - 1 groupe de test

puis on tourne, chaque groupe devenant groupe d'apprentissage et de test à son tour.

3.2 Validation (suite)

- Validité du modèle:
 - ◆ Nombre de cas corrects/nombre de cas total:
pour cela on réalise la Matrice de confusion:
 - Matrice à 2 dimensions dont l'élément (i,j) indique le nombre de cas de type i prédits par le modèle et observés comme étant réellement de type j sur les données de test
 - ◆ aussi variance et écart type (entre valeur prédite et la valeur observée)
- ➤ mesurer la validité d'un modèle est une tâche difficile

3.2 Validation: matrice de confusion

- Compare la situation réelle et la situation prédite par le modèle

Prédit ↓	Observé			Total
	Payé	Retard	Impayé	
Payé	80	15	5	100
Retard	1	17	2	20
Impayé	5	2	23	30
Total	86	34	30	150

- Sur les 86 factures payées, notre modèle en a prédit 80 correctement, et mal prédit 5 + 1
- Validité = nb exact (diagonale) / nb total = $80/150 = 0.53$

4. Utiliser le modèle

- Deux manières d'utiliser le modèle obtenu:
 - ◆ Voir les résultats du modèle sur les données
 - classes, règles du modèles,...
 - objectif: analyse
 - ◆ Appliquer le modèle pour prédire sur de nouvelles données
 - recommander des actions
 - ex. Accorder un crédit, analyse de risque, détection de fraude...

III- Outils et techniques de data mining

Typologie des méthodes

1) Selon les objectifs

2) Selon le type d'apprentissage:

- ◆ Apprentissage supervisé / non supervisé

3) Selon les types de modèles obtenus

- ◆ modèles prédictifs
- ◆ modèles descriptifs

Selon les objectifs

- Classification: consiste à examiner les caractéristiques d'un objet et lui attribuer une classe
 - ◆ attribuer ou non un prêt à un client
 - ◆ établir un diagnostic
- Prédiction: consiste à prédire la valeur future d'un attribut en fonction d'autres attributs
 - ◆ prédire la "qualité" d'un client en fonction de son revenu, de son nombre d'enfant,...

Selon les objectifs (suite)

- Association: consiste à déterminer les attributs qui sont corrélés
 - ◆ analyse du panier de la ménagère: ex. poisson et vin blanc
- Segmentation: consiste à former des groupes homogènes à l'intérieur d'une population
 - ◆ tâche souvent faite avant les précédentes pour trouver les groupes sur lesquels appliquer la classification.

Selon le type d'apprentissage

- Apprentissage supervisé:
 - ◆ processus dans lequel l'apprenant reçoit des exemples d'apprentissage comprenant à la fois des données d'entrée et de sortie
 - classification, prédiction
- Apprentissage non supervisé:
 - ◆ processus dans lequel l'apprenant reçoit des exemples d'apprentissage ne comprenant que des données d'entrée
 - Association, segmentation

Selon les types de modèles obtenus

- Modèles prédictifs: utilisent les données avec des résultats connus pour développer des modèles permettant de prédire les valeurs d'autres données
 - ◆ Exemple: modèle permettant de prédire les clients qui ne rembourseront pas leur crédit
 - ◆ classification, prédiction
- Modèles descriptifs: proposent des descriptions des données pour aider à la prise de décision. Les modèles descriptifs aident à la construction de modèles prédictifs
 - ◆ segmentation, Association

Qques méthodes pour construire les modèles

■ Classification:

- ◆ Arbres de décision
- ◆ K plus proches voisins
- ◆ Réseaux de neurones
- ◆ ...

■ Prédiction:

- ◆ Arbres de décision
- ◆ Réseaux de neurones
- ◆ ...

■ Association:

- ◆ Règles d'association
- ◆ ...

■ Segmentation:

- ◆ k moyennes (ou K-means)
- ◆ K plus proches voisins (K-nearest-neighbor)
- ◆ Réseaux de neurones
- ◆ ...

IV-

Classification / Prédiction

1. Arbres de décision

Présentation

- Arbre de décision= représentation graphique d'une procédure de classification

- Objectif:

Étant donné un ensemble de valeurs d'attributs (variable prédictive ou variable endogène)

- age
- type de véhicule

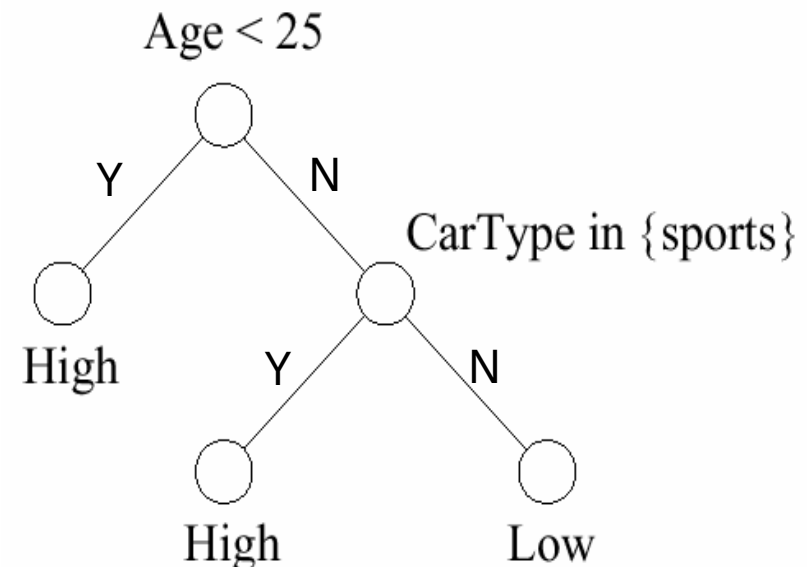
Prédire la valeur d'un autre attribut (variable cible ou variable exogène)

- risque

Exemple*

<i>rid</i>	Age	Car Type	Risk
0	23	family	High
1	17	sports	High
2	43	sports	High
3	68	family	Low
4	32	truck	Low
5	20	family	High

(a) Training Set



(b) Decision Tree

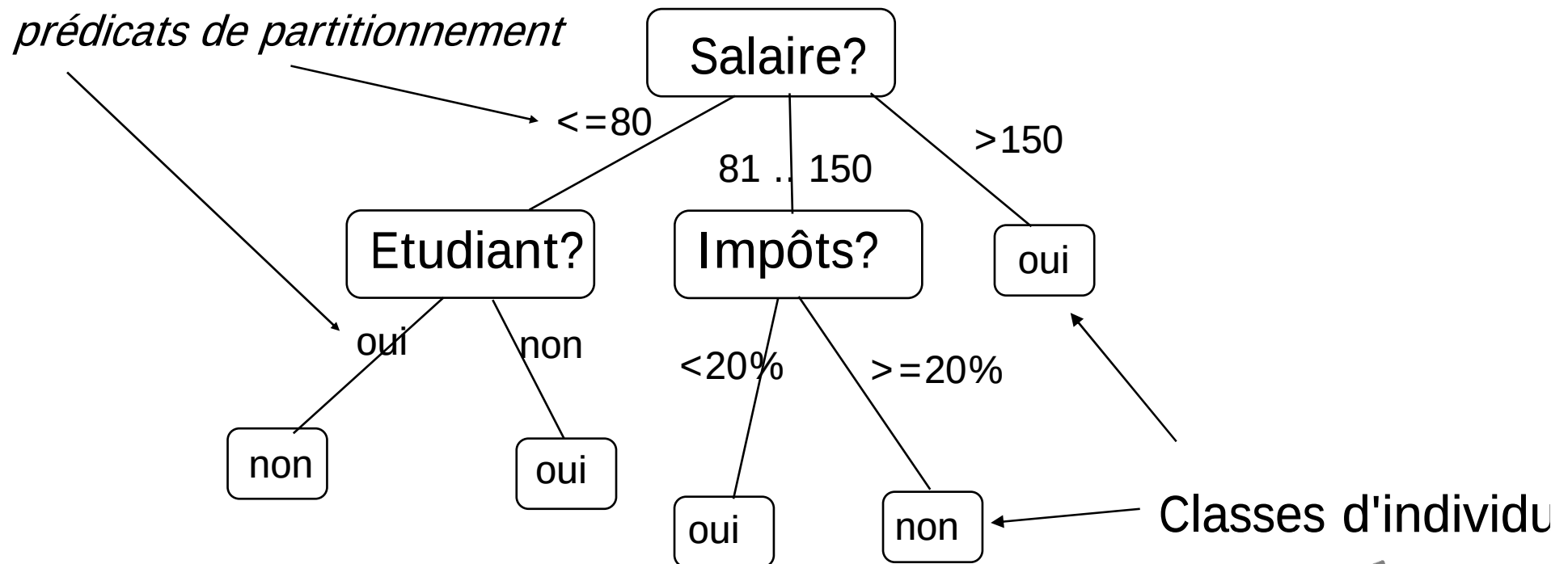
On cherche à prédire la valeur de l'attribut Risk
en fonction des attributs Age et Car Type

Présentation

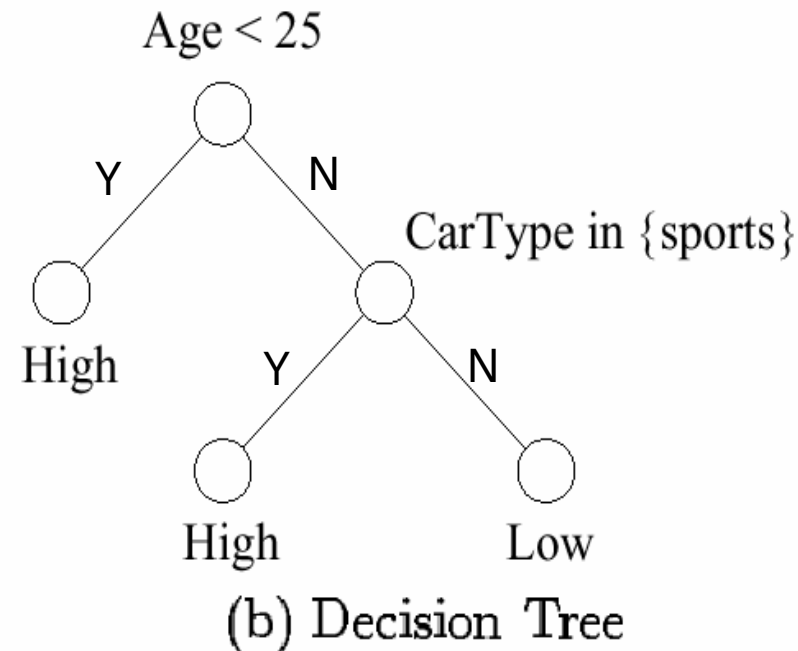
- Arbre
 - ◆ Un noeud représente une classe d'individus de plus en plus fine depuis la racine
 - ◆ un arc représente un prédicat de partitionnement de la classe source
- Arbre peut être interprété comme des règles de déduction

Exemple

- On cherche à prédire la valeur de la variable Contrôle impôt (oui ou non)



Règles



- Règles:
 - ◆ Si age < 25 alors High
 - ◆ Si age > 25 et CarType in {sports} alors High
 - ◆ Si age > 25 et CarType not in {sports} alors Low

Arbres

- Différents types d'arbre
 - ◆ binaire: 2 branches partent de chaque noeud
 - ◆ n-aire: n branches partent de chaque nœud
- Différents algorithmes
 - ◆ ID3 (Quilan 79)
 - ◆ CART (Brieman et al. 84)
 - ◆ Assistant (Cestnik et al. 87)
 - ◆ See5 (Quilan 97)
 - ◆ ...

Idée

- Arbre est construit en découpant successivement les données en fonction des variables prédictives
- Algorithme générique:

Segmenter (T)

si (tous les tuples de T appartiennent à la même classe de variable à prédire ou critère d'arrêt) **alors** retourner une feuille; 2

Pour chaque attribut prédictif A

évaluer la qualité de découpage selon A 1

Utiliser l'attribut donnant la meilleure découpe pour découper en sous-ensembles $T_1, \dots, T_n$ selon les valeurs de A 4

Pour i = 1..n *Segmenter (Ti)*;

Fin

Problèmes fondamentaux pour construire l'arbre

- 1) Choix de l'attribut discriminant
- 2) Affectation d'un label à une feuille
- 3) Arrêt de la segmentation (choix de la profondeur de l'arbre)
 - ◆ arbre trop profond n'est pas un bon arbre car trop complexe et trop adapté à l'ensemble d'apprentissage et donc pas assez généraliste
- 4) Choix des bornes de discrétisation
 - ◆ comment découper les valeurs d'un attribut continu, par exemple salaire : 1-25000 , 25000-50000, 50000-100000 ou 1-50000, 50000-100000?

Critère de sélection d'attributs

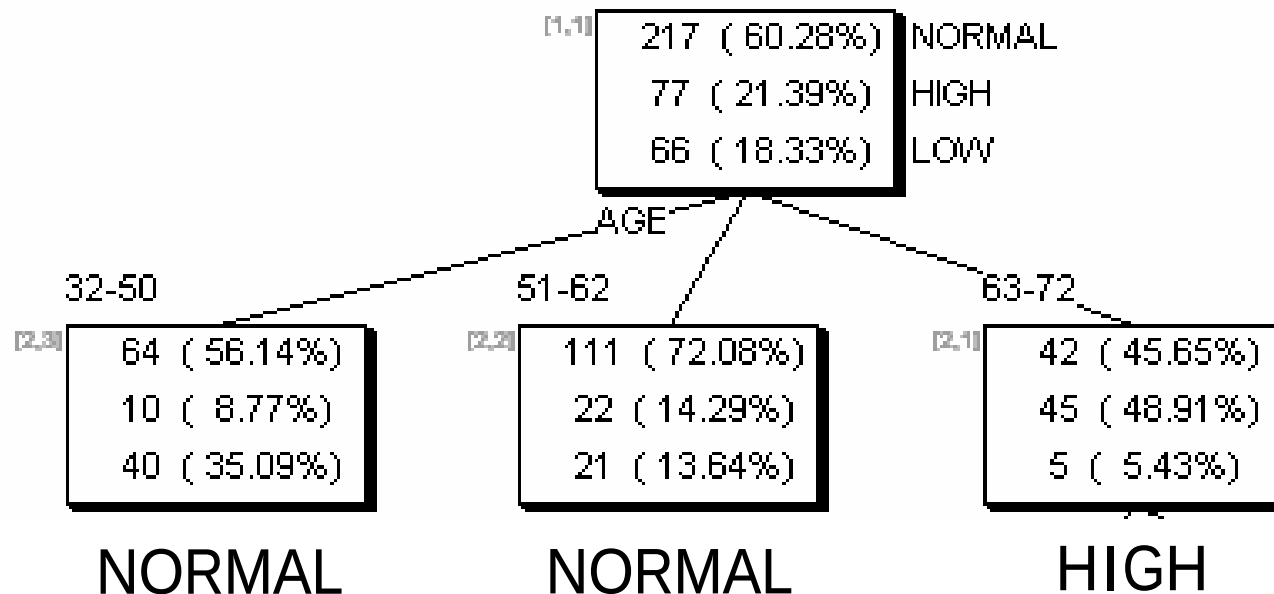
- Si y est l'attribut dont la valeur à prédire à partir des valeurs des attributs prédictifs x_i
 - ◆ choisir l'attribut dont la valeur est la plus "liée" (a le plus d'influence sur) avec celle de y
- Plusieurs techniques provenant de la théorie de l'information:
 - ◆ Ratio du gain ou entropie de Shannon
 - ◆ Index Gini
 - ◆ χ^2
 - ◆ ...

Ratio du gain / entropie

- On parle de gain d'information ou d'entropie (concepts inverses)
- On va chercher à choisir l'attribut qui va induire le gain d'information le plus élevé (ou dont l'entropie est la plus basse)
 - ◆ Intuitivement, l'entropie mesure le degré de désordre qui restera si on découpe selon cet attribut -> entropie la plus basse est la meilleure
- Donc pour chaque attribut candidat, on va calculer son entropie et on choisit celui qui a l'entropie la plus basse.

Affectation d'un label à une feuille

- On affecte la modalité la plus fréquente



Arrêt de la segmentation

- Différentes techniques:
 - ◆ pre-pruning
 - ◆ post-pruning
- pre-pruning: on arrête l'expansion de l'arbre selon certains critères:
 - ◆ profondeur maximale
 - ◆ effectif de chaque sous-groupe: on fixe un seuil (souvent empiriquement)
 - ◆ on calcule des mesures comme pour le choix de l'attribut de segmentation (e.g gain d'information, χ^2 , ...) auquel on associe un seuil en dessous duquel la segmentation sera refusée.

Arrêt de la segmentation

- post-pruning:
 - ◆ On laisse l'arbre se construire jusqu'au bout
 - ◆ On retire des sous-arbres de l'arbre :
 - A l'aide d'heuristiques ou grâce à l'intervention d'un expert, l'arbre est élagué tant que l'erreur de l'arbre élagué reste inférieure à celle de l'arbre non élagué.
 - ◆ Le nœud duquel on a retiré un sous-arbre devient une feuille et porte le label de la valeur la plus fréquente du sous-arbre

Choix des bornes de discrétisation

- On fixe les valeurs candidates comme les valeurs au milieu de deux valeurs consécutives
 - ◆ ex: 35, 45, 52...
 - ◆ -> 40, 48.5
- Puis on calcule éventuellement la meilleure valeur parmi celles là grâce à des mesures telles que le gain, le X^2 ...

Algorithme ID3

Classe C : valeur d'attribut à prédire
(ex: $C1$: risque=high, $C2$: risque=Low)
tuples : ensemble des tuples de l'échantillon
liste_attributs: ensemble des attributs

Procédure **Générer_arbre_décision**

Créer un nœud N

si $tuples$ est vide **alors**

retourner une feuille portant le label "Failure"

si $tuples$ sont dans la même classe C **alors**

retourner N avec comme label C

si $liste_attributs =$ vide **alors**

retourner N avec comme label le nom de la classe la plus fréquente dans l'échantillon

Sélectionner l'attribut a , le plus discriminant parmi $liste_attributs$

Affecter le label a au nœud N

Pour chaque valeur a_i de a

créer une branche issue du nœud N avec comme condition $a = a_i$

Soit t_i l'ensemble des éléments $tuples$ vérifiant cette condition

Attacher le nœud retourné par Générer_arbre_décision ($t_i, liste_attributs - a$)

Calcul de l'Entropie (1): Quantité d'info pour classifier l'exemple

- Soit S un ensemble de s tuples
- Soit C comprenant m valeurs différentes
 - ◆ définissant C_i classes ($i=1, \dots, m$)
- Soit s_i le nombre de tuples de S appartenant à C_i
- $I(s_1, \dots, s_m)$ = quantité d'information nécessaire pour classifier l'ensemble des tuples
- $I(s_1, \dots, s_m) = -\sum_{(i=1..m)} p_i \log_2(p_i)$
 - ◆ p_i : probabilité qu'un tuple appartienne à C_i
 - ◆ $p_i = s_i/s$

Calcul de l'Entropie (2): entropie de l'attribut

- Soit A un attribut candidat possédant v valeurs $\{a_1, \dots, a_v\}$.
- A permet de partitionner l'ensemble S en v sous-ensembles $\{S_1, \dots, S_v\}$
- S_i comprend les tuples ayant la valeur a_i pour A
- Soit s_{ij} le nombre de tuples du sous-ensemble S_j appartenant à C_i

$$E(A) = - \sum_{(j=1..v)} \left((s_{1j} + \dots + s_{mj}) / s \right) \times I(s_{1j}, \dots, s_{mj})$$

Pour chaque
valeur de A

Probabilité qu'un tuple ait cette
valeur ds ce ss-ensemble

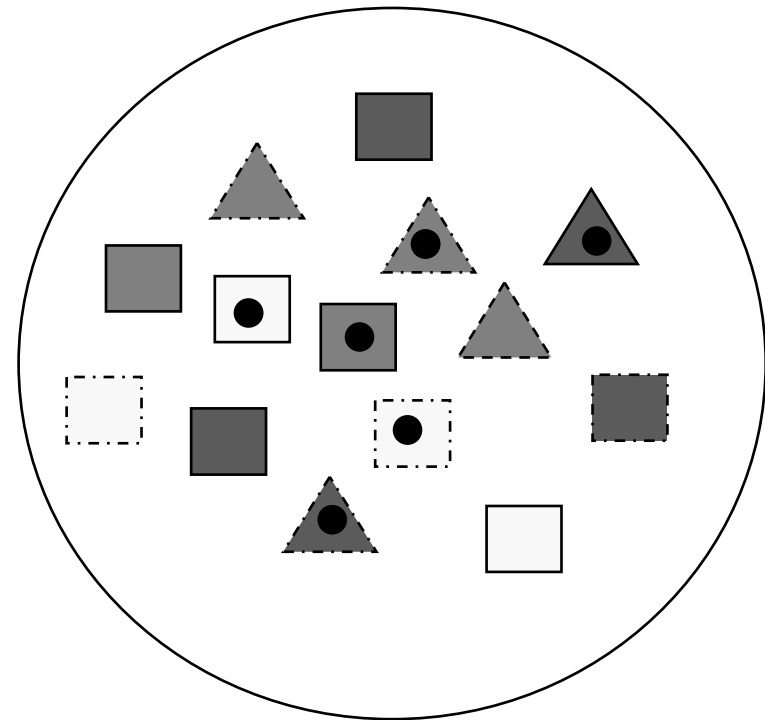
Qté d'info nécessaire pour
classifier les tuples ayant
cette valeur

Gain

- Gain = réduction d'entropie (i.e. de désordre) espérée si on utilise A
- On va donc choisir l'attribut ayant le gain le plus élevé
- Données:
 - ◆ Soit S un ensemble de s tuples
 - ◆ Soit C comprenant m valeurs différentes définissant C_i classes ($i=1,\dots,m$)
 - ◆ Soit s_i le nombre de tuples de S appartenant à C_i
 - ◆ Soit A l'attribut candidat
- $\text{Gain}(A) = I(s_1, \dots, s_m) - E(A)$

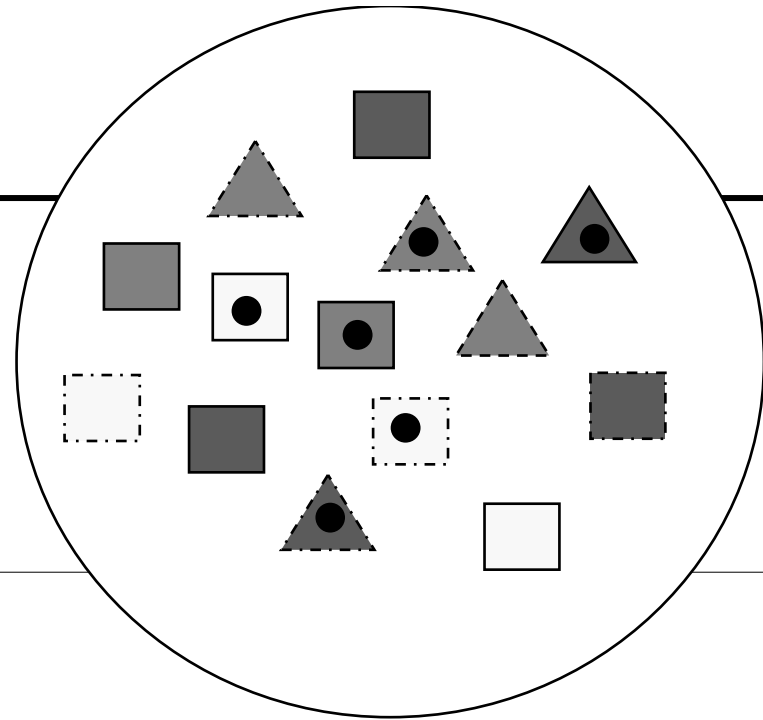
Exemple

n°	couleur	contour	point?	forme
1	vert	pointillé	non	triangle
2	vert	pointillé	oui	triangle
3	jaune	pointillé	non	carré
4	rouge	pointillé	non	carré
5	rouge	plein	non	carré
6	rouge	plein	oui	triangle
7	vert	plein	non	carré
8	vert	pointillé	non	triangle
9	jaune	plein	oui	carré
10	rouge	plein	non	carré
11	vert	plein	oui	carré
12	jaune	pointillé	oui	carré
13	jaune	plein	non	carré
14	rouge	pointillé	oui	triangle



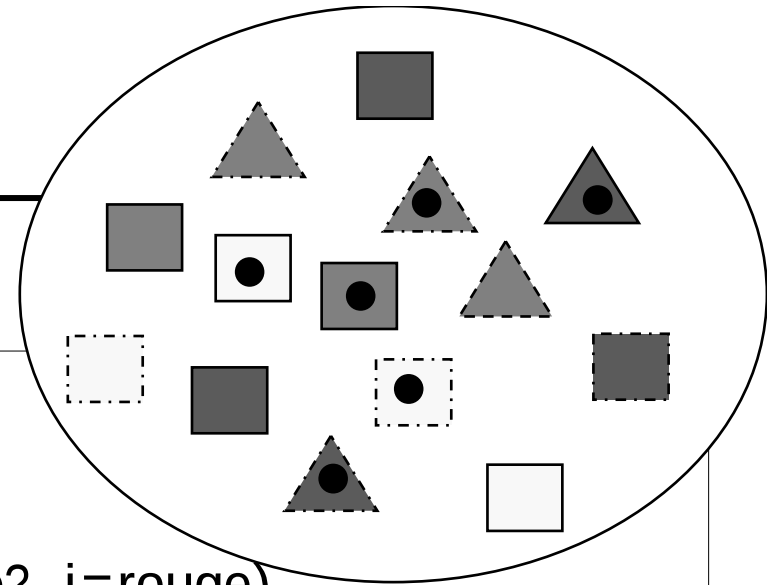
On veut prédire la forme

Exemple



- Il y a deux valeurs possibles:
 - ◆ carré ou triangle
 - ◆ Soit C1: carré, C2: triangle
 - ◆ C1 possède 9 élt, C2 5 élt
- Information nécessaire pour classifier l'exemple
 - ◆ $I(s_1, \dots, s_m) = -\sum_{(i=1..m)} p_i \log_2(p_i)$
avec s_i : nombre de tuples appartenant à C_i
- $I(s_1, s_2) = I(9, 5) = -9/14 \log_2 9/14 - 5/14 \log_2 5/14 = 0.940$

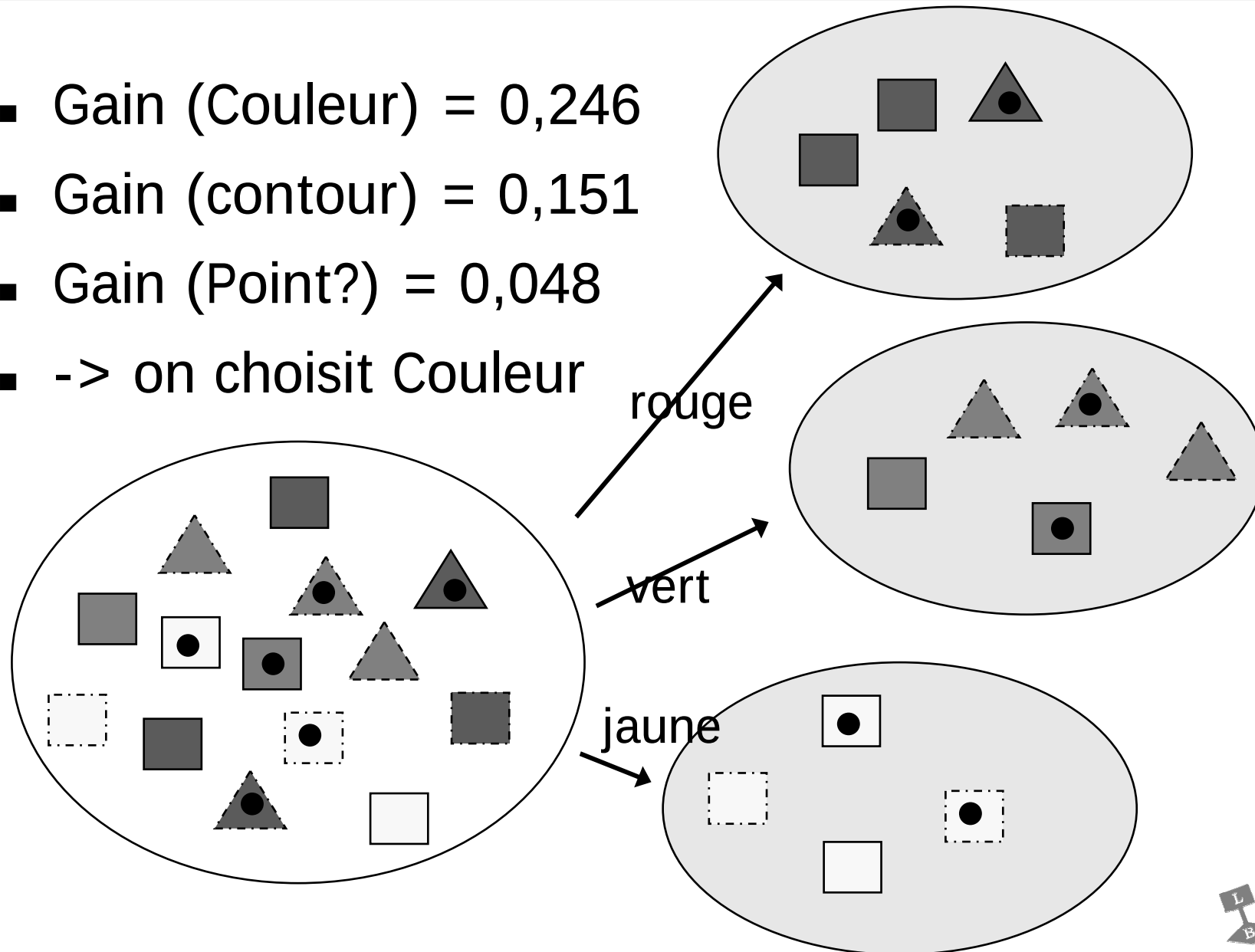
Exemple



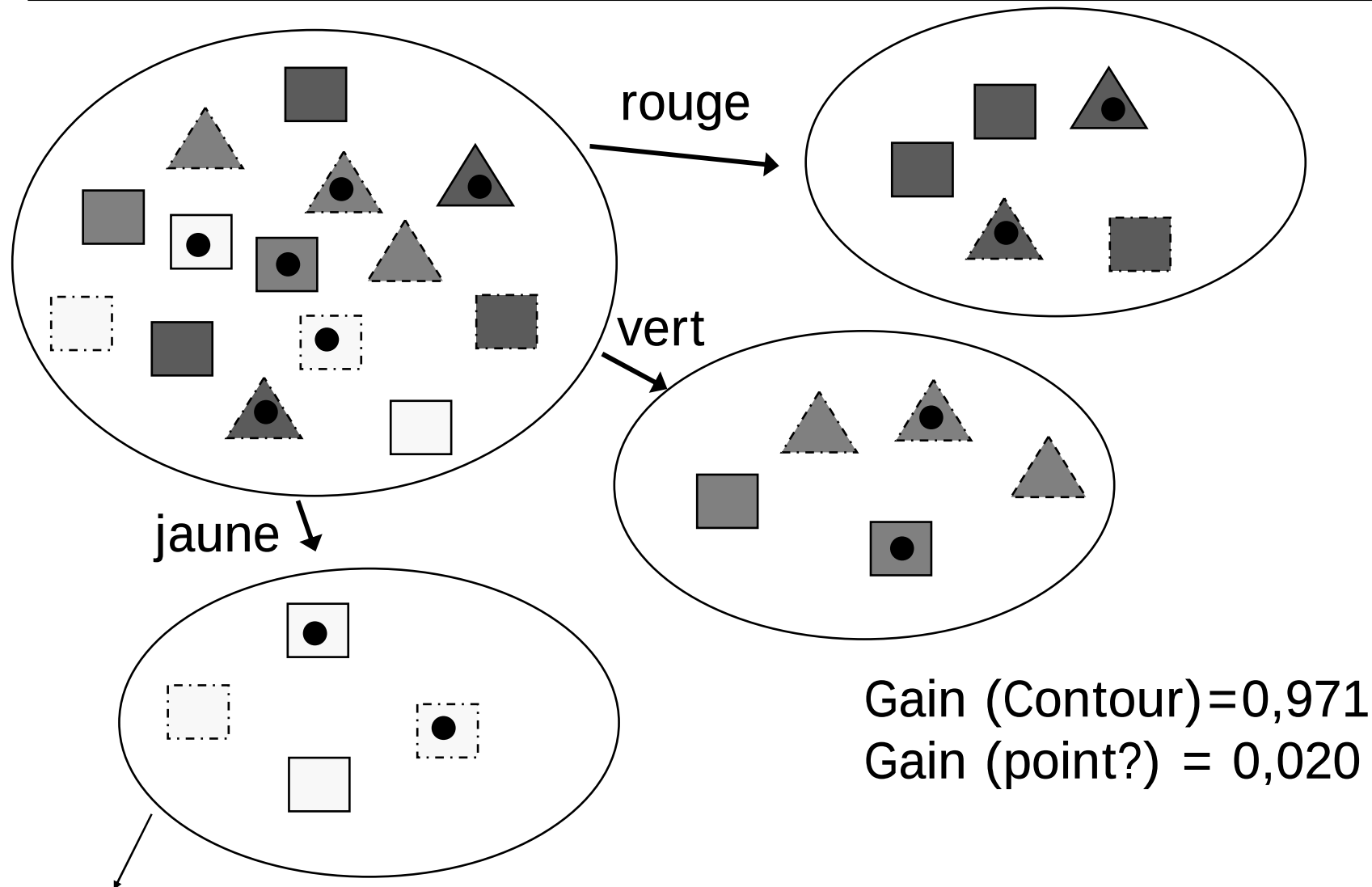
- Entropie de l'attribut Couleur?
 - ◆ C1: carré, C2: triangle
 - ◆ $E(A) = -\sum_{(j=1..v)} (s_{1j} + \dots + s_{ij})/s \times I(s_{1j}, \dots, s_{ij})$
- Pour couleur = rouge (i= carré ou triangle?, j=rouge)
 - ◆ $s_{11} = s_{\text{carré/rouge}} = 3$ $s_{21} = s_{\text{triangle/rouge}} = 2$
 - ◆ $I(s_{11}, s_{21}) = I(3, 2) = -3/5 \log_2 3/5 - 2/5 \log_2 2/5 = 0,971$ *5 rouges
- Pour couleur = vert
 - ◆ $s_{12} = s_{\text{carré/vert}} = 2$ $s_{22} = s_{\text{triangle/vert}} = 3$
 - ◆ $I(s_{12}, s_{22}) = I(2, 3) = -2/5 \log_2 2/5 - 3/5 \log_2 3/5 = 0,971$
- Pour couleur = jaune
 - ◆ $s_{13} = s_{\text{carré/jaune}} = 4$ $s_{23} = s_{\text{triangle/jaune}} = 0$
 - ◆ $I(s_{13}, s_{23}) = I(4, 0) = -4/4 \log_2 4/4 - 0/4 \log_2 0/4 = 0$
- $E(\text{couleur}) = 5/14 I(s_{11}, s_{21}) + 5/14 I(s_{12}, s_{22}) + 4/14 I(s_{13}, s_{23}) = 0,694$
- Gain (couleur) = $0,940 - 0,694 = 0,246$

Exemple

- Gain (Couleur) = 0,246
- Gain (contour) = 0,151
- Gain (Point?) = 0,048
- -> on choisit Couleur



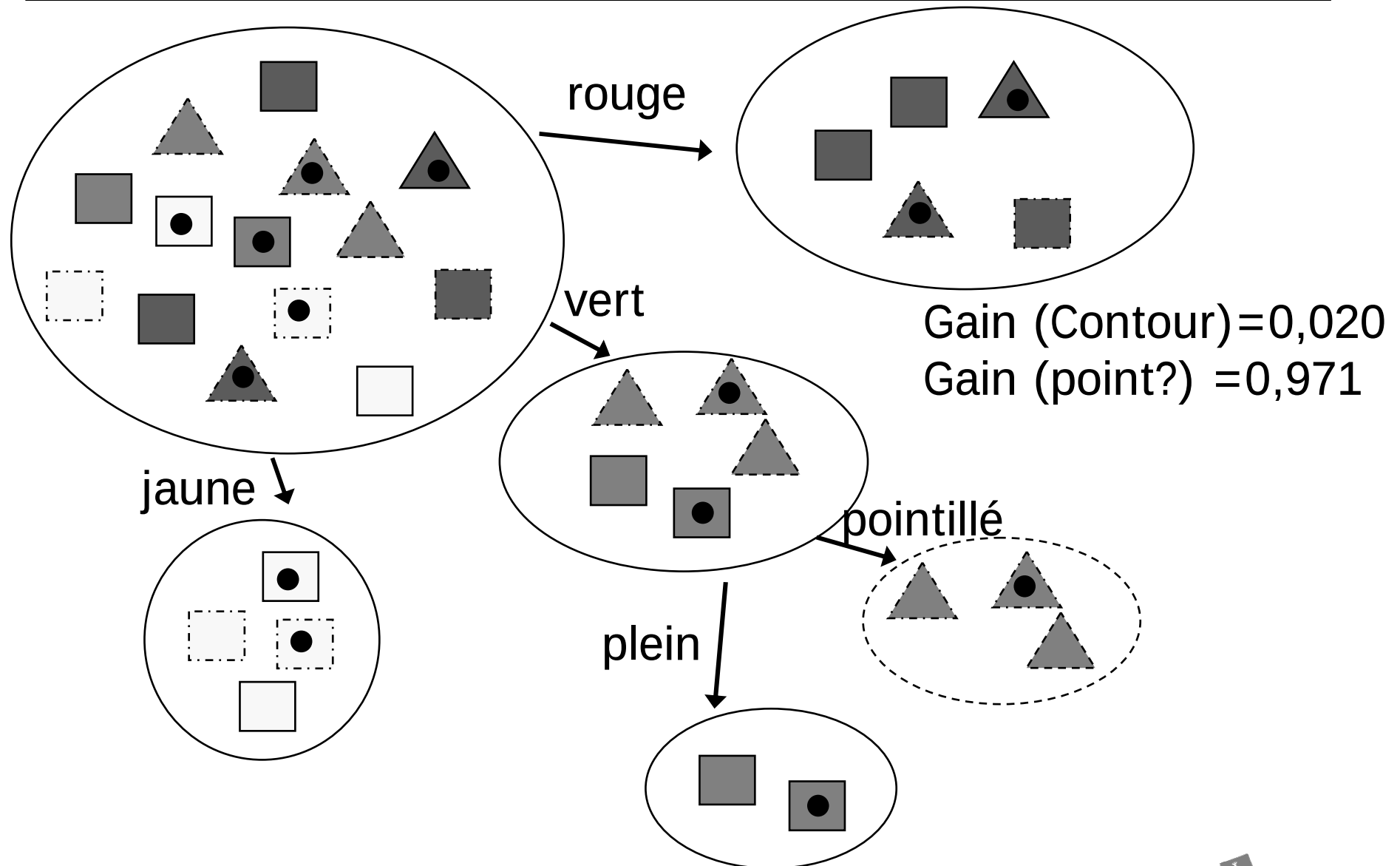
Exemple



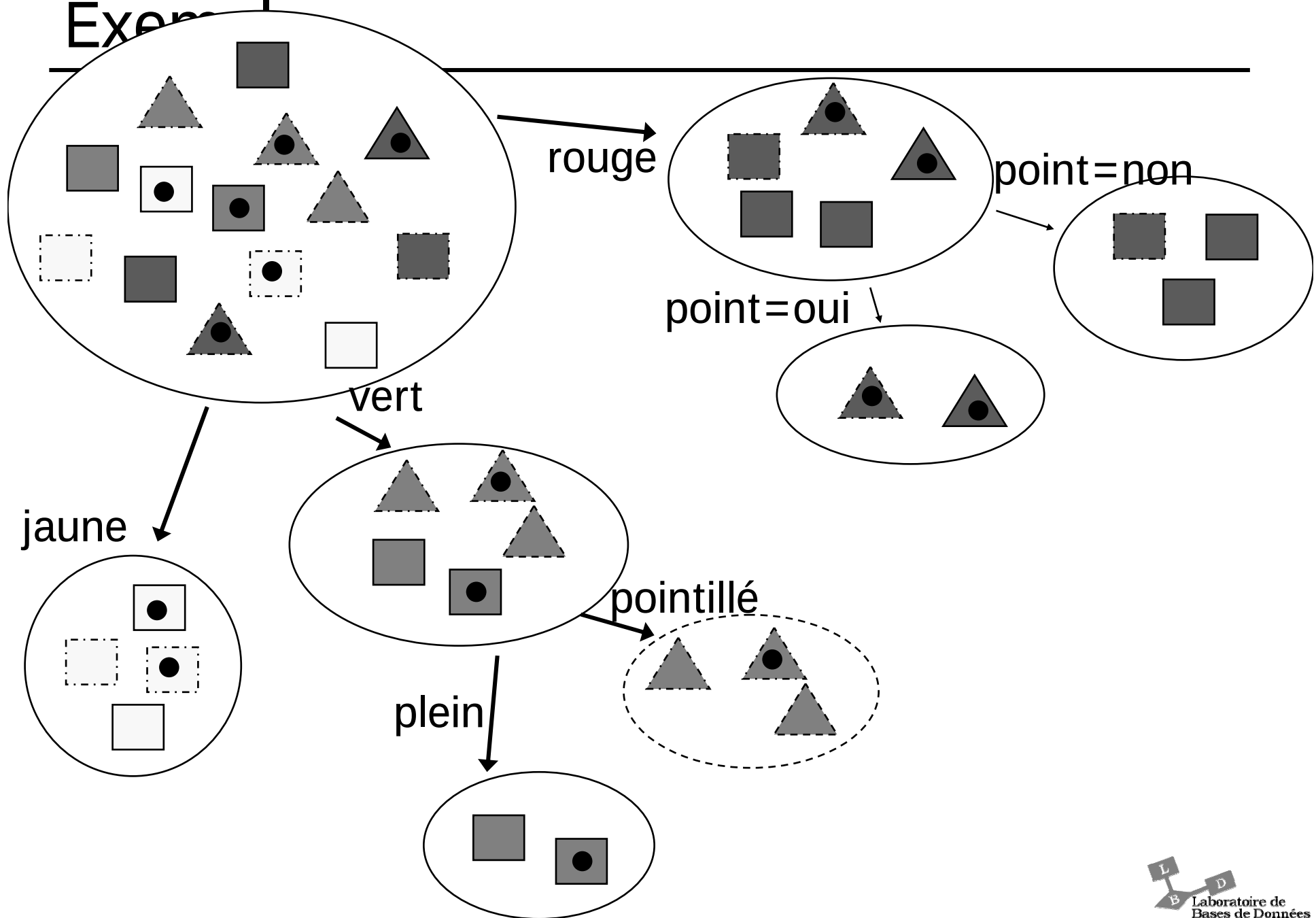
Gain (Contour)=0,971
Gain (point?) = 0,020

On s'arrête car ce sont tous des carrés
(tous les tuples dans la même classe)

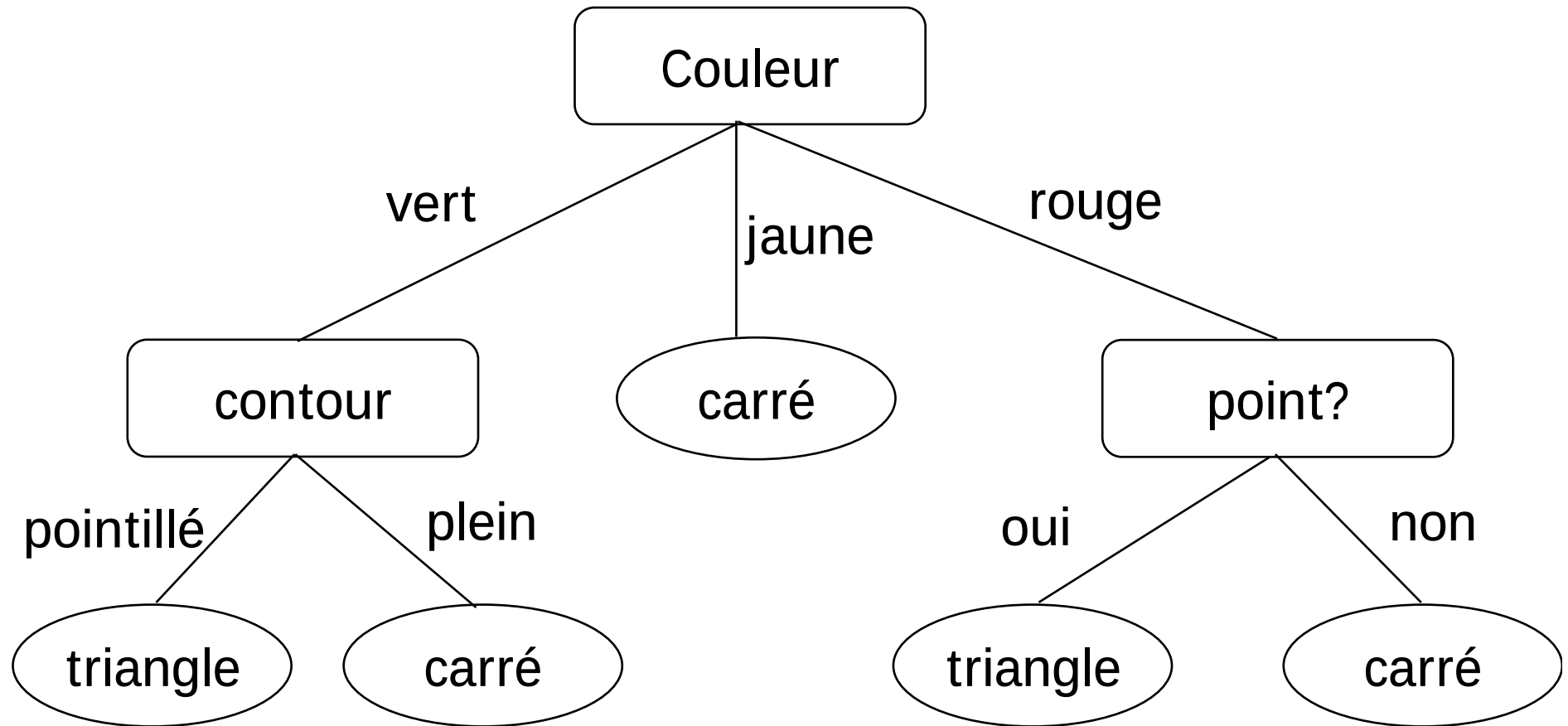
Exemple



Exemple 1



Arbre



IV- Classification

2. k-Plus proches voisins

k-Plus proches voisins

- Méthode de classification
- Idée: recherche de cas similaires au cas à résoudre et utilisation des décisions de ces cas proches pour choisir une décision
- On a:
 - ◆ un échantillon dont on connaît la classification
 - ◆ une fonction de distance
 - ◆ une fonction de choix de la classe en fonction de celle des plus proches voisins

Algorithme

fonction de distance

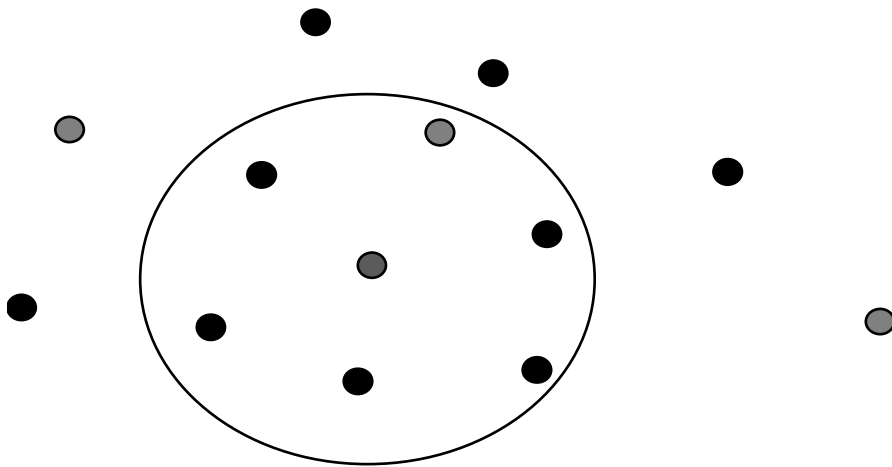
procédure **k-PPV** (*k nb de PPV, e élément à classifier*)

Déterminer les k plus proches voisins de e

Choisir la classe C en fonction de celle des k plus proches voisins

Retourner C

fonction de choix de la classe



Le point rouge est un nouveau cas à classer.

A l'aide de la fonction de distance, on calcule ses 6 ($k=6$) plus proches voisins. Ils sont dans le cercle rouge. Le point rouge sera classifié parmi les points noirs car c'est la classe majoritaire (fonction de choix) de ses plus proches voisins

Problèmes fondamentaux de K-PPV

- Fonction de choix de la classe
 - ◆ Choix de la classe majoritaire
 - ◆ Choix de la classe majoritaire pondéré:
 - L'appartenance à chaque classe d'un des k voisins est pondérée par la distance du point à ce voisin.
- Détermination de la fonction de calcul de la distance entre points
 - ◆ différents types d'attributs

Fonction de calcul de la distance

- Champs numériques:
 - ◆ $d(x,y) = |x-y|$
 - ◆ $d(x,y) = |x-y| / d_{\max}$ où $d_{\max}$ est la distance max entre 2 réels du domaine considéré
- Champs discrets:
 - ◆ données binaires: 0 ou 1.
 - $d(0,0)=d(1,1)=0$
 - $d(0,1)=1$
 - ◆ données énumératives
 - $d=1$ si valeurs égales 0 sinon
 - ◆ données énumératives ordonnées: distance en fonction de la relation d'ordre
- Champs complexes: distance $((x,y,z),(a,b,c))$
 - ◆ distance euclidienne : $d = d(x,a)^2 + d(y,b)^2 + d(z,c)^2$
 - ◆ sommation: $d = d(x,a) + d(y,b) + d(z,c)$
- D'autres bien plus compliquées...
 - ◆ Difficile et souvent applicatif: quelle est la distance entre bleu et vert?

V- Règles d'association

Règles

- Règle:
 - ◆ implication du type « SI x ALORS y » ou $x \Rightarrow y$
 - SI achat (Couche culotte) ALORS achat (lait maternisé)
 - SI achat (Pain) ET achat (Fromage) ALORS achat (Vin)
- Découverte de règles :
 - ◆ découverte de relations plus fines entre les données
 - ◆ recherche des associations peut s'appliquer sur :
 - l'ensemble des données
 - une donnée cible (la conclusion est donnée par l'utilisateur)
- Applications :
 - ◆ analyse des tickets de caisse (corrélations entre produits achetés, comportements de produits)
 - ◆ diagnostic de crédit, etc.

Règles – support – confiance

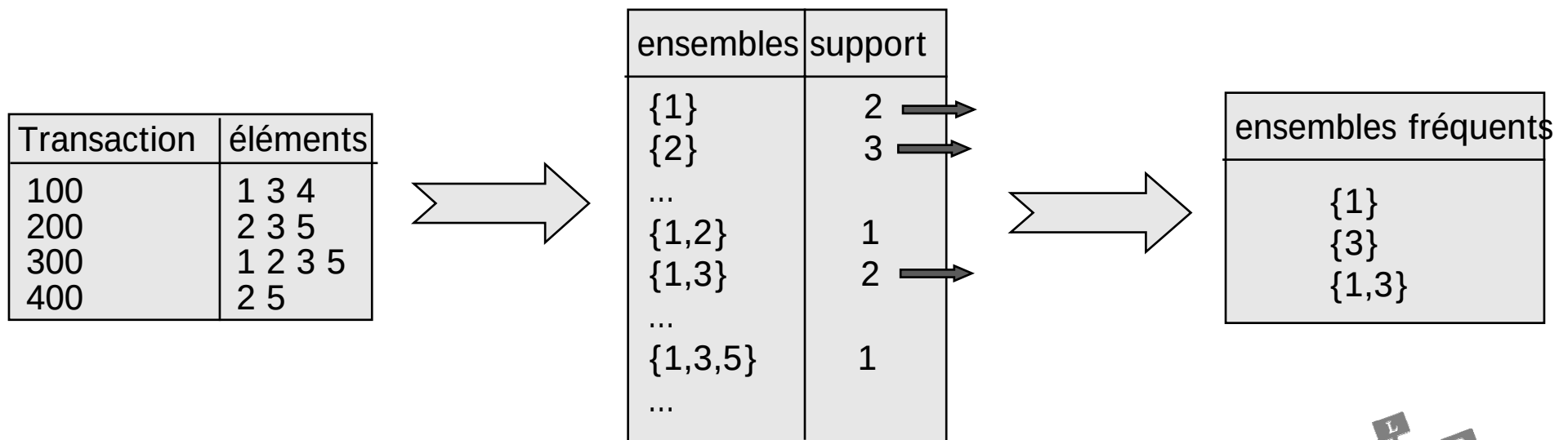
- Mais une règle n'est pas toujours vraie
 - ◆ deux indicateurs : support et confiance d'une règle
- Support: % de règles vérifiant la transaction
 - ◆ = Proportion d'objets contenant tous les attributs de la règle
 - ◆ $\text{Support}(XY) = |XY| / |BD|$
 - $|XY|$ nombre transactions comportant X et Y
 - $|BD|$ nombre transactions sur l'ensemble de la base
 - ◆ Ex: 5% des clients ont acheté des brosses à dent et du dentifrice
- Confiance: % de transactions vérifiant l'implication $X \Rightarrow Y$
 - ◆ $\text{Confiance}(X \Rightarrow Y) = |XY| / |X|$
 - ◆ $\text{Confiance}(XY) = \text{Support}(XY) / \text{Support}(X)$
 - ◆ Ex: Parmi les clients qui ont acheté des brosses à dent, 90% des clients ont également acheté du dentifrice

Extraction des règles

- Seules les règles qui ont un certain support et une certaine valeur de confiance sont intéressantes
 - ◆ valeurs de *minconf* et *minsup* doivent être fixées par l'analyste
- La plupart des méthodes pour extraire des règles, retiennent les règles dont le support est supérieur à *minsup* et parmi celles-ci retiennent celles dont la confiance est supérieure à *minconf*

Génération de règles (1)

- Idée: on calcule le support de tous les combinaisons d'objets possible de la base et on ne garde que celles dont le support est supérieur à un seuil donné = ensembles fréquents



Génération de règles (2)

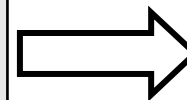
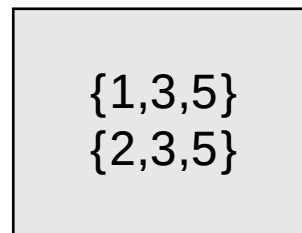
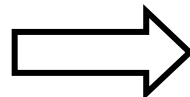
- Un ensemble de k éléments est appelé k -ensemble
- Lemme de base:
 - ◆ Tout k -ensemble fréquent est composé de sous-ensembles fréquents
 - ◆ 3-ensemble fréquent est composé de 2-ensembles et de 1-ensembles fréquents
- Procédure:
 - ◆ 1) on cherche les 1-ensembles fréquents
 - ◆ 2) on cherche les 2-ensembles fréquents
 - ◆ ...
 - ◆ n) on cherche les n -ensembles fréquents

jusqu'à ce qu'il n'y ait plus d'ensemble fréquent de taille supérieure

Génération d'un k-ensemble fréquent

- Génération:
 - ◆ A partir des 1-ensemble fréquents on génère des 2-ensembles fréquents,
 - ◆ A partir des 2-ensemble fréquents on génère des 3-ensembles fréquents,
 - ◆ ...
- Règle de génération: procédure apriori-gen
 - ◆ L'algorithme génère un candidat de taille k à partir de deux candidats de taille k-1 différents par le dernier élément
 - ◆ on garde ce candidat s'il ne contient pas de k-1 ensemble qui n'est pas fréquent
- Exemple:
 - ◆ {1,3,5}, {2,3,5}
 - ◆ On ne garde que {2,3,5} parce que {1,3,5} comprend le 2-ensemble {3,5} qui ne fait pas partie des 2-ensembles fréquents. Donc {1,3,5} n'est pas un 3-ensemble fréquent

2-ensemble	support
{1,3}	2
{2,3}	2
{2,5}	3
{1,5}	2



3-ensemble	support
{2,3,5}	2

Algorithme APriori

```
L1 = {frequent 1-ensemble} ;  
for (k = 2 ; Lk-1 ≠ ∅ ; k++) do {  
    Ck = apriori-gen(Lk-1); // Génération nouveaux candidats  
                                k-ensemble fréquents  
    for each transactions t ∈ DB do { // Comptage  
        Ct = { subset(Ck, t) }; // sous-ensembles de la BD  
        foreach c ∈ Ct do c.count++; } comptage support  
    Lk = { c ∈ Ck | c.count ≥ minsup } ; // Filtrer par rapport au  
                                           support  
}  
Answer = {Lk} ;
```

Exemple

Données en entrée

Transaction	éléments
100	1 3 4
200	2 3 5
300	1 2 3 5
400	2 5

Ck

Lk

lire BD pour
calculer
support

1-ensemble	support
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

filtrage
(support)

1-ensemble	support
{1}	2
{2}	3
{3}	3
{5}	3

Apriori-gen

2-ensemble
{1,2}
{1,3}
{1,5}
{2,3}
{2,5}
{3,5}

lire BD pour
calculer
support

2-ensemble	support
{1,2}	1
{1,3}	2
{1,5}	1
{2,3}	2
{2,5}	3
{3,5}	2

filtrage
(support)

2-ensemble	support
{1,3}	2
{2,3}	3
{2,5}	3
{3,5}	3

Apriori-gen

3-ensemble
{2,3,5}

lire BD pour
calculer
support

3-ensemble	support
{2,3,5}	2

filtrage
(support)

3-ensemble	support
{2,3,5}	2

Génération des règles

- A partir des k-ensembles fréquents:
 - ◆ pour chaque ensemble fréquent l , générer tous les sous-ensembles s non vides
 - ◆ pour chaque sous-ensemble s , générer la règle " $s \Rightarrow l - s$ " si $\text{Confiance}(s \Rightarrow l - s) > \text{minconf}$
- Exemple:
 - ◆ $\{2,3,5\}$: sous-ensembles non vides: $\{2\}$, $\{3\}$, $\{5\}$, $\{2,3\}$, $\{2,5\}$, $\{3,5\}$
 - ◆ $\text{Confiance}(X \Rightarrow Y) = \text{Support}(XY) / \text{Support}(X)$
 - ◆ règles:
 - $2 \text{ et } 3 \Rightarrow 5$ $\text{Confiance} = 2/2 = 100\%$
 - $2 \text{ et } 5 \Rightarrow 3$ $\text{Confiance} = 2/3 = 66\%$
 - $3 \text{ et } 5 \Rightarrow 2$ $\text{Confiance} = 2/2 = 100\%$
 - $2 \Rightarrow 3 \text{ et } 5$ $\text{Confiance} = 2/3 = 66\%$
 - $3 \Rightarrow 2 \text{ et } 5$ $\text{Confiance} = 2/3 = 66\%$
 - $5 \Rightarrow 2 \text{ et } 3$ $\text{Confiance} = 2/3 = 66\%$
 - ◆ Si seuil = 70% on ne garde que les règles 1 et 3

Transaction	éléments
100	1 3 4
200	2 3 5
300	1 2 3 5
400	2 5

Génération de règles redondantes

- Pour éviter la génération de règles redondantes:
 - ◆ R1) Il suffit de considérer le k-ensemble fréquent le plus grand:
 - si il n'y a pas de règles dont la confiance $>$ seuil, on considère les sous-ensembles
 - ◆ R2) Il suffit d'extérioriser les règles de condition minimale
 - $2 \text{ et } 3 \Rightarrow 5$
 - $2 \Rightarrow 3 \text{ et } 5$
 - on extériorise que: $2 \Rightarrow 3 \text{ et } 5$
 - on ne garde que les règles dont la confiance $>$ seuil

Inconvénients de Apriori

1. Beaucoup de candidats
 - ◆ 10^4 1-ensembles vont générer 10^7 2-ensembles
2. Plusieurs scans de la BD (pour calculer les supports)
 - ◆ on doit faire $n+1$ scans pour trouver les n -ensembles fréquents
3. Les indicateurs de support et confiance sont insuffisants
 - ◆ support est symétrique :
 - il ne distingue pas $A \Rightarrow B$ de $B \Rightarrow A$
 - ◆ confiance favorise les événements rares:
 - $\text{confiance}(A \Rightarrow B) = \text{Support}(AB) / \text{Support}(A)$
 - si $\text{Support}(A)$ est faible, $\text{Support}(AB)$ aussi et confiance $\uparrow$

Améliorations

-> **1** Algorithme AprioriTID qui garde en mémoire les n° de tuples correspondant à chaque k-ensemble

- ♦ Une lecture de BD pour les 1-ensemble, ensuite pour chaque niveau k on fait l'intersection entre les k-1 ensembles:

- Lecture 1-ensemble, on place en mémoire :

$\{A\}$: tuples 1, 3, 5 , 9

$\{B\}$: tuples 3, 5, 11, 123

- Génération 2-ensemble : $\{A,B\} : \{1, 3, 5 , 9\} \cap \{3, 5, 11, 123\} =$
tuples 3,5

-> **2** Marquer les tuples qui ne contiennent pas de k-ensembles fréquents pour qu'ils ne soient pas lus lorsqu'on considère les j-ensembles fréquents ($j > k$)

-> **3** Autres mesures:

- ♦ conviction : $P(X) * P(\text{non } Y) / P(X, \text{non } Y)$
- ♦ "lift": $\text{Confiance } (X \Rightarrow Y) / \text{Support}(Y)$

Autres algorithmes

- Apriori partitionné
- Algorithme de comptage dynamique
- FP-Growth
- Close

VI-Segmentation

k-moyennes (k-mean)

Méthode de segmentation

- Méthode permettant de découvrir les groupes (clusters) d'individus similaires en se basant sur une mesure de similarité (ou distance) pour grouper les données
- On ne connaît pas les classes à priori
 - ◆ elles sont à découvrir automatiquement
- Buts :
 - ◆ Maximiser la similarité intra-classes et minimiser la similarité inter-classes.
- Méthodes par partitionnement:
 - ◆ Construire k partitions et les corriger jusqu'à obtenir une similarité satisfaisante

Méthode des K-moyennes

- Méthode des K-moyennes (*MacQueen'67*)
 - ◆ Choisir (au hasard) K éléments qui sont initialement déclarés "centres" des K groupes
 - ◆ Pour tous les autres objets, les placer dans le groupe de centre le plus proche (basé sur la fonction de distance)
 - ◆ Recalculer le centre de gravité de chaque groupe
 - ◆ Itérer l'algorithme jusqu'à ce que les objets ne changent plus de groupe
- C'est l'Algorithme le plus utilisé

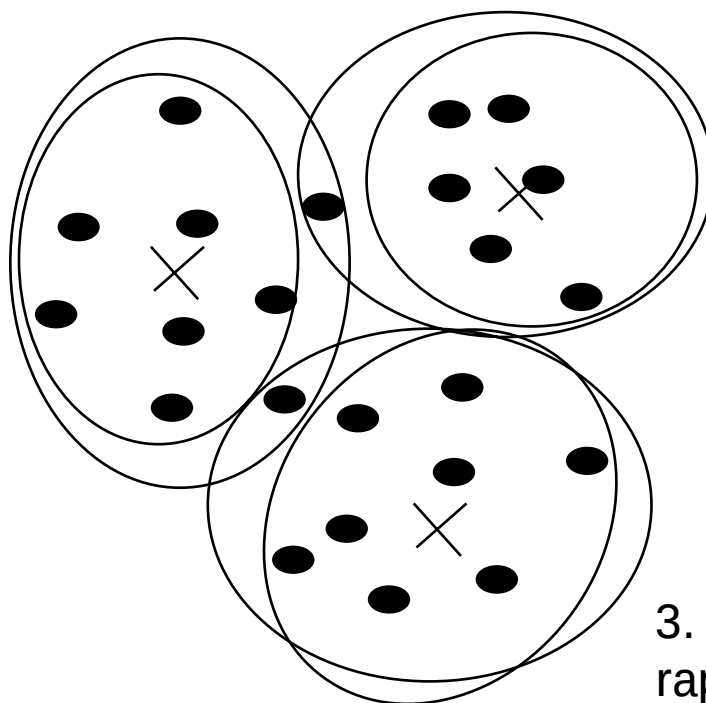
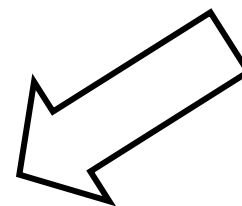
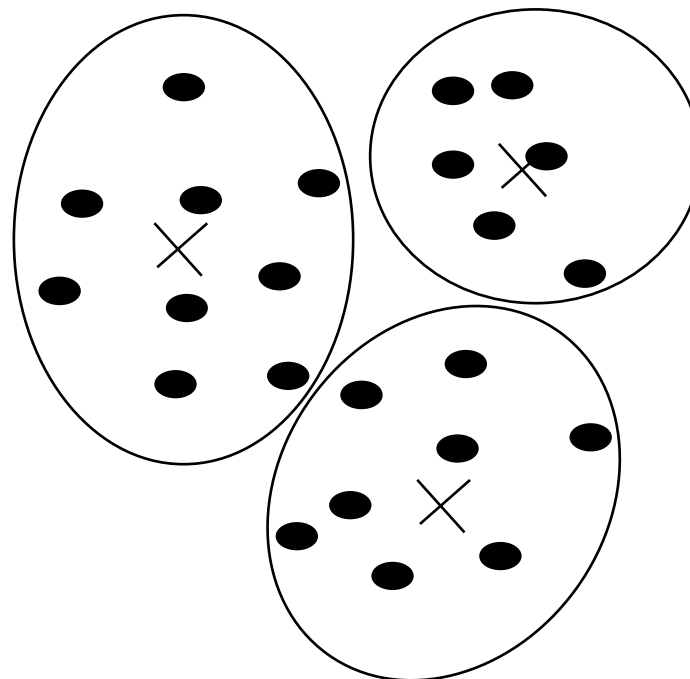
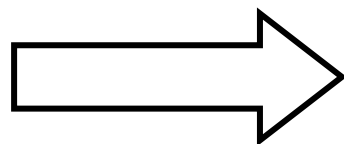
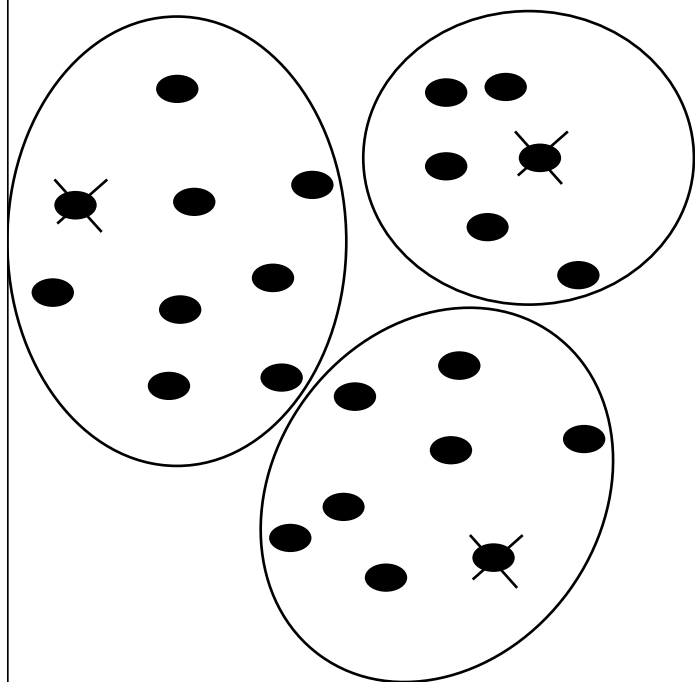
Algorithme

- **K-mean** (k clusters, n objects):
 - ◆ choisir aléatoirement k objets comme centres de cluster
 - ◆ assigner chaque objet au centre le plus proche
 - ◆ recalculer les k centres
 - ◆ tant que les k centres ont été changés
 - réassigner les tuples
 - recalculer les k centres

2. recalcul des centres

1. clusters initiaux

3. reconstruction des clusters par rapport aux nouveaux centres



Variantes de l'algorithme k-moyennes

- Différents choix de fonction de calcul de distance (comme pour KPPV)
- Choix des centres initiaux

Autres méthodes de segmentation

- méthode hiérarchique par agglomération
 - ◆ Chaque individu représente un groupe
 - ◆ Trouver les deux groupes les plus proches
 - ◆ Grouper ces deux groupes en un nouveau groupe
 - ◆ Itérer jusqu'à ce que tous les objets soient dans un groupe ou on a le nombre k (paramètre de l'algo) de groupes recherchés

Références

- Transparents de Ricco Rakotomalala
 - ◆ <http://chirouble.univ-lyon2.fr/~ricco/cours/index.html>
 - ◆ cette page regroupe aussi pleins de liens très intéressant sur le data mining (cours, transparents, logiciels...)
- Transparents de M.A. Aufaure
- Transparents de G. Gardarin
 - ◆ <http://perso.wanadoo.fr/georges.gardarin/Cours2003.htm>
- Support de cours de R. Gilleron et M. Tommasi
 - ◆ <http://www.grappa.univ-lille3.fr/polys/fouille/index.html>
- Livres
 - ◆ "Internet/intranet et Bases de données" G. Gardarin
 - ◆ "Data mining, Concepts and techniques" J. Han, M. Kamber
 - ◆ "Introduction to data mining" Two Crows Corporation

Références - logiciel

- Sipina
 - ◆ <http://chirouble.univ-lyon2.fr/~ricco/cours/index.html>
 - ◆ gratuit
 - ◆ installé en IN2
- Tanagra
 - ◆ <http://chirouble.univ-lyon2.fr/~ricco/cours/index.html>
 - ◆ gratuit
- Weka
 - ◆ installé en IN2
 - ◆ gratuit
- DBMINER
 - ◆ installé en IN2